

Profesor: Borja Rey Seoane

TEMA 5

INTRODUCCIÓN A ANGULAR

Módulo: Desenvolvimento Web en Contorna Cliente

Ciclo: DAW

Introdución a Angular

Neste tema aprenderemos os conceptos básicos sobre o *framework* Angular e centraremos tamén na creación de aplicacións que sigan o patrón MVC e a funcionalidade Cross-Browser¹, para o cal pretendemos acadar os seguintes obxectivos:

- Identificar as diferenzas que presenta o modelo en función dos navegadores.
- Programar aplicacións Web de xeito que funcionen en navegadores con diferentes implementacións do modelo.
- Independizar o contido, o aspecto e o comportamento en aplicacións Web.



Arquitectura de Angular 2+

Angular 2+ é un *framework* para crear aplicacións Web de páxina única (SPA²) baseado en TypeScript e nun modelo orientado a compoñentes. Fornece ferramentas para estruturar a interface, xestionar datos e eventos, inxectar dependencias e traballar de forma modular e testable.

Principios clave

- **Modelo orientado a compoñentes:** a UI divídese en pezas reutilizables con clase (lóxica), modelo (HTML) e estilos (CSS).
- **Data binding:** fluxo de datos entre clase e vista (tanto unidireccional como bidireccional).
- **Inxección de dependencias:** creación e provisión de servizos onde se precisan.

¹ Denominamos *Cross Browser* a aquelas aplicacións Web que foron desenvolvidas para verse e funcionar exactamente igual en calquera navegador.

² *Single Page Application:* aplicacións Web que posúen un único documento HTML sobre o que se van producindo os sucesivos cambios de interface.

- **Modularidade:** organización en unidades independentes e reempregables.
- **Testabilidade e mantenibilidade:** separación clara de responsabilidades.

Pezas da arquitectura

- **Compoñentes.**
- ***Templates e data binding.***
- **Directivas.**
- **Servizos e inxeccións de dependencias.**
- ***Pipes.***
- **Módulos e modo *standalone*.**
- **Enrutamento.**
- **Asincronía e control de estado.**
- **Detección de cambios e renderizado.**
- **Formularios reactivos.**

TypeScript

TypeScript (TS) é un superconxunto³ de JavaScript. Como características máis salientables atopamos:

- Foi desenvolvido por Microsoft para actuar coma unha capa adicional de JS ES6.
- Engade tipado estático e funcionalidade orientada á obxectos.
- Posúe unha sintaxe moderna (clases, interfaces, decoradores) que se transpila a JavaScript estándar.
- O proceso de compilado axuda a identificar erros de programación con maior facilidade que o interpretado de JS.

Ideas clave para favorecer a nosa transición dende JS:

- Anotacións de tipo (opcionais): engadimos `:tipo` a variables e parámetros.

```
let nome: string = "Lara";  
let idade: number = 20;  
function suma(a: number, b: number): number { return a + b; }
```

³ Un superconxunto, en programación, é unha linguaxe que inclúe completamente a outra e engade funcionalidade extra.

- Inferencia de tipos: a miúdo non temos que indicar os tipos (aínda que se recomenda), senón que TS dedúceos.

```
let activo = true;           // boolean
let contadores = [1, 2];    // number[]
```

- Posúe unha serie de tipo elementais e compostos habituais:

```
string, number, boolean, null, undefined, any (evitase), unknown, void.
Arrays: number[], string[].
Unións e opcionais:
let id: number | string;
function cargar(id?: number) { /* id pode vir ou non */ }
```

- Permite implementar *interfaces* para describir obxectos:

```
interface Videoxogo {
  titulo: string;
  ano: number;
  plataforma?: string; // opcional
}

let v: Videoxogo = { titulo: "Celeste", ano: 2018 };
```

- Manexa clases (igual que en JS moderno, pero con tipos):

```
class Persoa {
  constructor(public nome: string, private idade: number) {}

  saudar(): string { return `Ola, son ${this.nome}`; }
}
```

- Permite realizar importacións de módulos, bibliotecas e fontes de datos externas:

```
import { Component } from '@angular/core';
```

Exemplo T05E01

Este exemplo introduce a sintaxe básica de TypeScript nun proxecto mínimo: partimos dun ficheiro .ts con código sinxelo (tipos básicos, funcións e, opcionalmente, unha interface/clase) e comprobamos como se compila a .js para executarse no navegador.

O exemplo persegue os seguintes obxectivos didácticos:

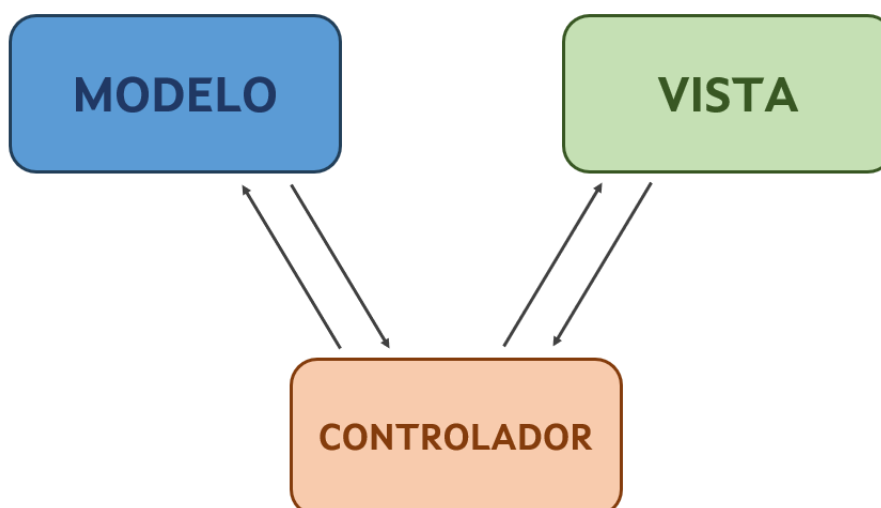
- Entender que é TypeScript e por que «todo JS é TS», pero non á inversa.
- Transpilar código .ts a .js (tsc) e interpretar o papel de tsconfig.json.

- Empregar tipado básico (`number`, `string`, `boolean`), anotacións en parámetros e valores de retorno.
- Distinguir erros de tipado en tempo de desenvolvemento fronte a erros en execución.
- Cargar o `.js` xerado nun HTML e validar o funcionamento.

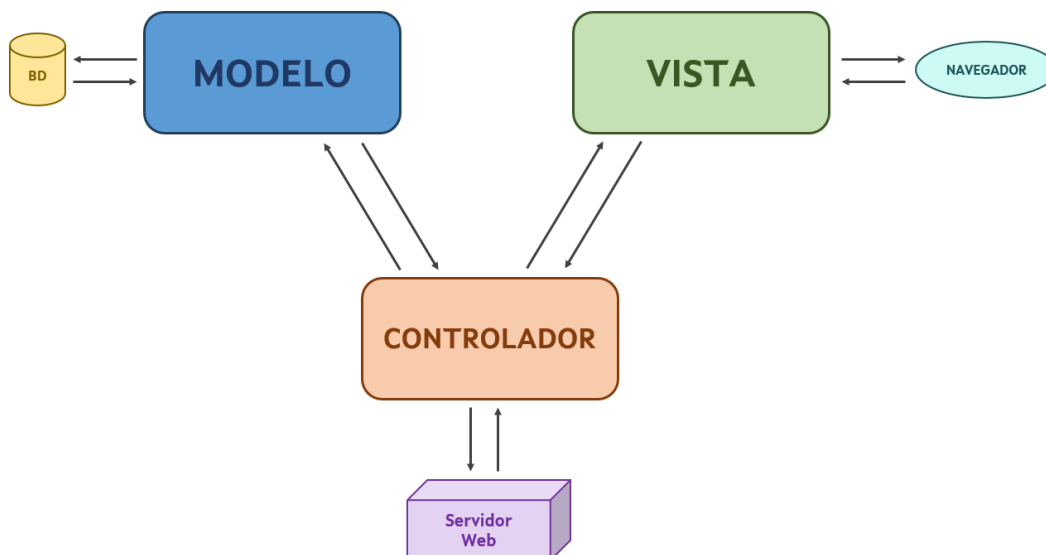
O patrón MVC

O MVC ou **Modelo-Vista-Controlador** é un patrón de arquitectura de *software* que separa a lóxica dunha aplicación da súa vista. Así, distinguimos:

- **Modelo:**
 - É a parte encargada da xestión dos datos e a lóxica de negocio.
 - Comunícase co controlador para fornecer información á vista.
 - Polo xeral traballa sobre algún tipo de base de datos.
- **Vista:**
 - É a representación visual dos datos, todo o que teña que ver coa interface de usuario e as interaccións.
 - Recibe do controlador a información procesada procedente do modelo.
 - Tanto o modelo como o controlador non se preocupan na forma en que os datos serán amosados, recaendo esta responsabilidade unicamente na vista.
- **Controlador:**
 - É a parte encargada de xestionar e recibir as ordes do usuario, actualizar o modelo e comunicar os cambios á vista.
 - Actúa de intermediario entre o modelo e a vista.



O diagrama típico do patrón MVC.



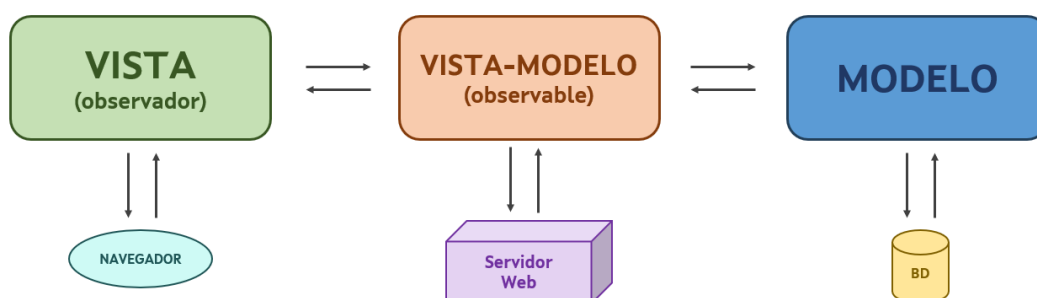
O diagrama do patrón MVC aplicado á programación Web.

MVC en Angular

Angular non segue o patrón MVC clásico, xa que está baseado en compoñentes que integran vista e lóxica. Así, en Angular temos:

- **Modelo:** son as clases e servizos que xestionan datos e negocio, independentes da interface.
- **Vista:** corresponde ao HTML dos compoñentes, incluíndo o data binding.
- **Controlador:** é a lóxica da compoñente, isto é, o código TypeScript que asume funcións tanto de vista como de controlador.

Este modelo adoita describirse como **MVVM (Modelo-Vista-Vista-Modelo)**, no que o *ViewModel* integra a lóxica de interacción entre modelo e vista, ou de forma máis xeral como **MVW (Modelo-Vista-Whatever)**, dada a flexibilidade na separación de responsabilidades.



O diagrama do patrón MVVM aplicado á programación Web.

Non obstante, tamén existe o modelo como lóxica de negocio, como poden ser os servizos ou todo aquilo que se inxecte e que si está totalmente independizado da vista.

Instalación e posta a punto

Neste capítulo trataremos os procedementos de instalación das ferramentas necesarias para desenvolver baixo Angular en Ubuntu 24.04 LTS:

- **Node.js:** contorna de execución JS de código aberto que permite executar o código fora de navegadores Web. Adoita empregarse no desenvolvemento en lado servidor.
- **Angular CLI:** ferramenta de liña de comandos que nos permite crear, desenvolver, estruturar e manter aplicacións Angular.

Instalación de Node.js

Para poder instalar a versión máis recente de **Node.js** en Ubuntu 24.04 primeiramente temos que seguir os pasos do sitio oficial da tecnoloxía:

<https://deb.nodesource.com/>

Con ese fin, dende a terminal actualizamos a listaxe de paquetes:

```
sudo apt update
```

Seguidamente, instalamos Node.js dende os repositorios oficiais de Ubuntu:

```
sudo apt install nodejs -y
```

Se quixeramos comprobar que a versión instalada é a correcta, faremos:

```
node -v
```

Velaquí o resultado no noso caso:

```
administrador@UBUNTUDAW:~$ node -v  
v18.19.1
```

Finalmente, instalamos o xestor de paquetes de NodeJS (npm):

```
sudo apt install npm -y
```

Podemos tamén comprobar a versión que acaba de instalar con:

```
npm -v
```


No noso caso:

```
administrador@UBUNTUDAW:~$ npm -v  
9.2.0
```

Instalación de Angular CLI

Unha vez instalado **Node.js**, temos xa dispoñible o Node Package Manager (ou simplemente, **npm**), un xestor de paquetes que vai permitirnos engadir funcionalidade ao noso servidor.

O primeiro paso é limpar a caché de npm:

```
sudo npm cache verify
```

Agora instalamos a versión de Angular CLI que desexamos (neste caso a 18.2.12):

```
sudo npm install -g @angular/cli@18.2.12
```

Igualmente ca no caso anterior, podemos comprobar a versión unha vez instalada para asegurar que é a correcta:

```
ng version
```

Obtendo algo semellante a isto:

```
Angular CLI  
  
Angular CLI: 18.2.12  
Node: 18.19.1  
Package Manager: npm 9.2.0  
OS: linux x64  
  
Angular:  
...  
  
Package                                Version  
-----  
@angular-devkit/architect              0.1802.12 (cli-only)  
@angular-devkit/core                   18.2.12 (cli-only)  
@angular-devkit/schematics             18.2.12 (cli-only)  
@schematics/angular                   18.2.12 (cli-only)
```

Primeiros pasos

Neste capítulo veremos os pasos a seguir para crear un primeiro proxecto de Angular ao que, posteriormente, engadiremos novas compoñentes.

No noso caso, empregaremos un novo cartafol chamado `Proxectos`, dentro do cal crearemos todos os nosos proxectos (se ben isto non é obrigatorio, e os mesmos poden ser situados en calquera lugar que desexemos).

Creación dun proxecto Angular

Situados no cartafol de proxectos, procedemos a crear un novo proxecto `ola-mundo` mediante a seguinte orde:

```
ng new OlaMundo
```

Se quixeramos crear unha aplicación modular, no canto de *standalone*, é dicir, tal e coma se traballaba nas versións anteriores de Angular, teríamos que empregar o seguinte comando:

```
ng new ola-mundo --no-standalone
```

Se quixeramos que non se crearan os ficheiros de probas unitarias para cada novo elemento TS da aplicación engadiríamos:

```
ng new ola-mundo --skip-tests
```

A continuación vánsenos presentar un par de preguntas:

- **Queres habilitar o autocompletado?**
 - Respostar SI a esta opción implica que os comandos de Angular CLI se autocompletarán ao premer TAB na terminal (coma calquera comando ordinario de GNU/Linux).

```
administrador@UBUNTUDAW:~/Proxectos$ ng new OlaMundo
Would you like to enable autocompletion? This will set up your terminal so pressing TAB
while typing Angular CLI commands will show possible options and autocomplete arguments.
(Enabling autocompletion will modify configuration files in your home directory.) [Y/n]
```

- **Queres compartir datos de uso do proxecto con Google?**

- Respostar SI implicará que eses datos usaranse coma *feedback* para a resolución de *bugs* e o desenvolvemento de novas versións do *framework* por parte da equipa de Angular.

```
Would you like to share pseudonymous usage data about this project with the Angular Team at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more details and how to change this setting, see https://angular.dev/cli/analytics.
```

```
(y/N) █
```

- **Que tipo de estilos queremos empregar?**

- Isto vai depender da tecnoloxía visual escollida (CSS, SCSS, Sass Indentado ou Less) aínda que, no noso caso, ficaremos con CSS estándar.

```
? Which stylesheet format would you like to use? (Use arrow keys)
> CSS [ https://developer.mozilla.org/docs/Web/CSS ]
Sass (SCSS) [ https://sass-lang.com/documentation/syntax#scss ]
Sass (Indented) [ https://sass-lang.com/documentation/syntax#the-indented-syntax ]
Less [ http://lesscss.org ]
```

- **Queres habilitar o prerrenderizado *Server-Side Rendering* (SSR) e *Static Site Generation* (SSG)?**

- SSR (*Server-Side Rendering*) é o proceso de renderizar unha aplicación de lado cliente completamente no servidor, no canto de no navegador. Só o resultado final do renderizado é enviado ao navegador do cliente. Isto pode mellorar o SEO da nosa páxina, pero tamén incrementa a carga do servidor.
- SSG (*Static Site Generation*) é o proceso de crear un sitio Web como conxunto de ficheiros HTML estáticos, no canto de como unha aplicación dinámica. Isto é útil se o noso sitio non require actualizacións frecuentes nin implica moita interacción co usuario.

```
? Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? (y/N)
```

Estrutura dun proxecto Angular

A resultas da creación do proxecto mediante a orde vista no punto anterior (`ng new`) aparecerá un directorio co nome do proxecto dentro do cal distinguiremos os seguintes elementos:

- **Cartafol `node_modules`:**

- Contén todas as dependencias de terceiros instalados mediante `npm`.
- Non se edita directamente, senón que as súas dependencias son especificadas a través do ficheiro `package.json`.

- **Cartafol `public`:**

- Contén imaxes e outros elementos que servirán como ficheiros estáticos para o servidor de desenvolvemento.
- Os elementos contidos nel serán copiados á aplicación definitiva durante a compilación.

- **Cartafol `src`:**
 - Constitúe o directorio principal do noso proxecto no que poderemos atopar os ficheiros de código fonte.
- **Ficheiro `angular.json`:**
 - É o ficheiro de configuración principal do proxecto.
 - Controla aspectos como a compilación, o *linting*⁴, as probas e a configuración dos estilos e *scripts*.
- **Ficheiro `package.json`:**
 - Contén a listaxe de dependencias do proxecto e *scripts* de comando.
 - Consta de tres seccións clave:
 - `dependencies`: dependencias precisas para executar a aplicación.
 - `devDependencies`: dependencias precisas para o desenvolvemento.
 - `scripts`: comandos para xestionar o proxecto.
- **Ficheiro `server.ts`:**
 - Punto de entrada para o servidor que executa a aplicación Angular no lado do servidor.
 - Funcións:
 - Configura o servidor HTTP.
 - Configura o renderizado SSR usando o `renderModule` de Angular.
 - Serve recursos estáticos como imaxes ou ficheiros CSS.
 - Define rutas para servir as vistas da aplicación.
- **Ficheiro `tsconfig.json`:**
 - Define as opcións do compilador, como o soporte a ECMAScript, a configuración dos módulos e as rutas aos distintos ficheiros.

O cartafol `src`

No interior deste cartafol atoparemos:

- **Cartafol `app`:**
 - Raíz de todos os módulos, compoñentes e servizos que iremos creando dentro do noso proxecto.
 - Dentro deste último cartafol atoparemos unha serie de ficheiros que nos serán de moita relevancia:
 - **`app.component.html`:**
 - Representa o contido HTML da compoñente principal da aplicación, sendo o elemento base empregado por Angular para realizar a ligazón de datos (*data binding*).

⁴ O *linting* é o chequeo automático do código na procura de erros programáticos e estilísticos.

- **app.component.css:**
 - Representa os estilos CSS da nosa compoñente principal.
- **app.component.ts:**
 - É o arquivo de TypeScript fundamental da compoñente principal, xa que inclúe a lóxica do mesmo. A páxina pódese entender coma unha unha árbore de compoñentes, onde a principal actúa coma raíz.
- **app.component.spec.ts:**
 - Trátase do ficheiro que permite configurar os test de proba unitarios na compoñente principal. Nesta actividade non imos empregalo pero é importante saber que fariamos uso deste tipo de arquivos mediante Angular CLI a través do comando:

```
ng test
```

- **app.module.ts:**
 - Este ficheiro TypeScript inclúe as dependencias da nosa Web. Ten coma obxecto a importación dos módulos necesarios, a declaración das compoñentes a empregar e a compoñente principal que será arrincada (este ficheiro non aparece en proxectos *standalone*, que é o xeito por defecto de traballo de Angular dende a versión 17).
- **app.config.server.ts:**
 - Contén a configuración específica para o lado servidor (define o comportamento da aplicación cando se executa no servidor).
 - Almacena:
 - URLs de APIs específicas para o servidor.
 - Garda a configuración de seguridade.
 - Garda configuracións de módulos e servizos que só son precisos no SSR.
- **app.config.ts:**
 - Contén a configuración xeral da aplicación, independentemente de se executa no lado do cliente ou do servidor.
 - Actúa coma punto central para a xestión de parámetros, provedores e outras configuracións globais.
 - Almacena:
 - URLs de APIs xerais.
 - Configuración de temas ou estilos.
 - Proveedores globais e módulos comúns.

- **app.routes.ts:**
 - Define as rutas da aplicación Angular, as cales especifican como se navega entre as distintas vistas e compoñentes.
 - Almacena:
 - As definicións das rutas principais.
 - A configuración das rutas con parámetros ou rutas dinámicas.
 - A configuración de *lazy loading* para cargar módulos de xeito diferido (en aplicacións modulares).
- **Cartafol assets:**
 - Almacena recursos estáticos coma imaxes, fontes, vídeos, ficheiros de configuración etc.
 - Os ficheiros contidos no mesmo serán incluídos na compilación e accesibles dende a URL base.
 - Dende Angular 18 este cartafol non é creado por defecto (aínda que pode crearse manualmente), podéndose empregar para tal función o cartafol `public` do directorio raíz do proxecto.
- **Ficheiro index.html:**
 - Ficheiro HTML principal da aplicación, é o punto de entrada onde Angular insire a súa vista a través da carga da compoñente raíz (`app.component`).
- **Ficheiro main.server.ts:**
 - Actúa coma punto de entrada da aplicación Angular no lado do servidor.
 - En aplicacións modulares inicializa o módulo raíz da aplicación para SSR.
 - Integra a aplicación Angular co servidor HTTP que executará a mesma.
- **Ficheiro main.ts:**
 - Ficheiro TS principal dende o que Angular arrinca a aplicación e arrinca o módulo raíz (en aplicacións modulares).
- **Ficheiro styles.css:**
 - Ficheiro de estilos globais da aplicación.
 - Todas as compoñentes herdan por defecto estes estilos.

Despregamento de aplicacións Angular

O despregamento dunha aplicación Angular implica poñela en funcionamento nun servidor ou contorna específica para ser accesible por usuarios ou para probas internas. Este proceso, coma o de calquera outro proxecto de *software* profesional, inclúe dúas fases principais:

1. **Contorna de desenvolvemento:** proba e visualización da aplicación localmente durante o desenvolvemento.
2. **Contorna de produción:** creación dunha versión optimizada da aplicación para a súa distribución final e explotación.

A estrutura modular de Angular facilita o despregamento tanto en fase de desenvolvemento como en produción, aproveitando ferramentas integradas en Angular CLI.

Contorna de desenvolvemento

O **despregamento en contorna de desenvolvemento** permite probar a aplicación en tempo real namentres se realizan cambios no código. Angular fornece un servidor de desenvolvemento a través do comando `ng serve`, que recompila automaticamente os cambios e os aplica á aplicación en tempo de execución.

A continuación veremos os comandos básicos relacionados con esta fase:

- Para **executar a aplicación no servidor de desenvolvemento** lanzaremos, dende o cartafol raíz do proxecto, o seguinte comando:

```
ng serve
```

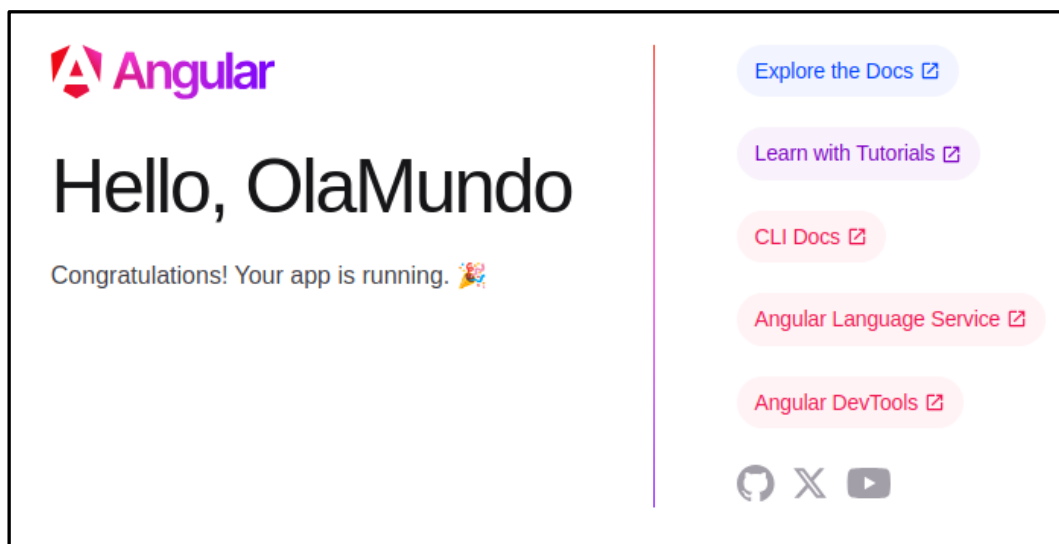
- Se ademáis queremos **abrir automaticamente a aplicación no navegador por defecto** lanzaremos:

```
ng serve -o
```

- Se quixeramos **indicar un porto distinto do porto por defecto** (4200) usariámos:

```
ng serve --port 4300
```

Por defecto, a pantalla de benvida dun novo proxecto en Angular 18 é a seguinte:



Contorna de produción

O **despregamento en contorna de produción** implica xerar unha versión optimizada da aplicación lista para ser servida a través dun servidor Web definitivo. Isto conséguese empregando o comando `ng build`, que produce una aplicación HTML5, CSS3 e JS contida no cartafol `dist`.

Estes son os comandos fundamentais empregados nesta fase:

- Para **compilar⁵ a aplicación** executamos, no directorio raíz do proxecto:

```
ng build
```

- Para **producir unha versión de produción** executamos:

```
ng build --configuration production
```

- Tamén podemos **especificar o cartafol base da aplicación** (útil para subdirectorios):

```
ng build --base-href="/mi-aplicacion/"
```

Os ficheiros xerados no cartafol `dist` pódense subir a calquera servidor Web que soporte ficheiros estáticos, como por exemplo Apache, Nginx, Firebase Hosting ou GitHub Pages.

O despregamento de aplicacións Angular require coñecer as diferenzas entre as contornas de desenvolvemento e produción. A través de comandos sinxelos e ferramentas integradas, Angular facilita a creación e distribución de aplicacións optimizadas para distintas necesidades. Adaptar a configuración ás particularidades do proxecto pode garantir mellores resultados tanto en probas como en produción.

Inicio e carga dun proxecto en Angular

Unha vez creado o proxecto (con `ng new`) e lanzado no servidor (con `ng serve`) procedemos a entender como está cargando esa páxina de inicio por defecto. Con ese fin, o servidor de Angular tomará os seguintes pasos:

1. **Carga do ficheiro `index.html`:** o navegador carga o documento raíz e atopa o elemento anfitrión da aplicación (por defecto, `<app-root>`). Este elemento é o «punto de montaxe» onde Angular inserirá a UI. Angular renderiza por defecto no DOM do navegador.
2. **Execución do `script main.ts (bootstrap)`:** executa `bootstrapApplication(...)`, o cal inicializa a aplicación en modo *standalone* e liga a compoñente raíz ao elemento anfitrión de

⁵ En realidade «transpilar», xa que o que faremos con esta acción será traducir a aplicación TypeScript ao seu equivalente JavaScript.

`index.html` (nos proxectos actuais non hai `AppModule`; o *bootstrap* faise directamente sobre a compoñente raíz).

3. **Creación e carga da compoñente raíz (`AppComponent`):** créase a compoñente raíz, aplícanse o seu *template* e estilos, e renderízase no `<app-root>`. A partir de aquí, actívase o ciclo de detección de cambios e a UI queda sincronizada co estado da aplicación.

Optimizacións no proceso de carga

- **Lazy loading:** permite cargar módulos de forma diferida, reducindo o tempo de inicio inicial.
- **Preloading:** mellora a experiencia cargando módulos en segundo plano.

A nosa primeira aplicación Angular

Para introducirmos ao desenvolvemento con Angular imos crear un primeiro proxecto e, como non pode ser doutro xeito, faremos o clásico **Ola Mundo!**:

1. Accedemos ao arquivo `app.component.html`:
1. Borramos todo o seu contido (podemos deixar unicamente a etiqueta `router-outlet` se queremos conservar o enrutado dinámico de Angular –do que falaremos posteriormente-).
2. Incorporamos simplemente unha cabeceira de exemplo:

```
<h1>Ola mundo!</h1>
<router-outlet></router-outlet>
```

3. Visualizando a nosa aplicación no navegador veremos o resultado da modificación anterior:

Ola mundo!

Exemplo⁶ T05E02

Este exemplo presenta a mostra máis básica de aplicación Angular, un «Ola Mundo!».

O exemplo persigue os seguintes obxectivos didácticos:

- Comprender a estrutura dun proxecto Angular básica.
- Identificar a compoñente raíz e as súas partes.
- Comprender como se relacionan entre si as distintas partes dunha compoñente.

⁶ Para probar os exemplos de Angular introducidos neste tema precisas crear un novo proxecto con estilos CSS e sen SSR habilitado. Logo, simplemente, copias o contido do ZIP ao directorio do proxecto, sobreescribindo os ficheiros coincidentes.

Enlazado de datos

O **enlazado de datos**, **enlace de datos** ou **data binding** é un dos conceptos fundamentais de Angular, e permite definir a comunicación entre as compoñentes e o DOM, asinxelando a definición de aplicacións interactivas sen necesidade de nos preocupar da carga e recepción de datos.

Hai diversos xeitos de enlazar datos en Angular (algúns mesmo posúen variantes), diferenciándose segundo o xeito no que se produce o fluxo de datos en dous tipos

- **Enlazado unidireccional.**
- **Enlazado bidireccional.**

Enlazado unidireccional

O **enlazado unidireccional** ou **one-way binding** posibilita o fluxo de datos entre compoñentes e DOM nun único sentido. Distinguimos, segundo ese sentido sexa, dous tipos:

- **Dende a compoñente ao DOM**, velaquí as dúas formas de traballo que hai:
 - Interpolación de *strings*: engade o valor dunha propiedade dende a compoñente, para o cal emprégase a estrutura `{{valor}}`. Por exemplo:

```
<li>Nome: {{ usuario.nome }}</li>
<li>E-mail: {{ usuario.email }}</li>
```

- Enlazado de propiedades: o valor pásase dende a compoñente á propiedade especificada, que pode ser simplemente un atributo HTML. Neste caso emprégase a estrutura `[propiedade]="valor"`. Vexamos un exemplo:

```
<input type="email" [value]="usuario.email">
```

- **Dende o DOM á compoñente**, conta cun único mecanismo:
 - Enlazado de eventos: chama a un método da compoñente cando se produce un evento DOM (clic, cambio, soltar unha tecla, etc.). Emprega a estrutura `(evento)="método"`. Por exemplo:

```
<button (click)="prepararAlmorzo()"></button>
```

Exemplo de interpolación de *strings*

Supoñamos un exemplo de TypeScript (podemos facer uso de `app.component.ts`) no que declaramos unha variable de tipo *String* contendo unha certa cadea e outras variables numéricas.

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'interpolacion-strings';
  numero1: number = 12;
  numero2: number = 20;
}
```

Agora, facendo uso da dobre chave podemos substituír o nome desas variables polo seu valor e mesmo facer operacións con elas (como é o caso da operación de multiplicación que observamos no código). Así quedaría o noso código HTML (`app.component.html`).

```
<h1> {{ title }} </h1>
<p>O resultado de multiplicar {{ numero1 }} por {{ numero2 }} é {{ numero1*numero2 }} </p>
```

Obténdose un resultado semellante ao seguinte:

Nome do Proxecto (accedido con data-binding)
A multiplicación de num1*num2 é: 30

Exemplo T05E03

Este exemplo presenta o uso da interpolación de *strings* para enviar información dende o controlador (TS) ao modelo (HTML).

O exemplo persigue os seguintes obxectivos didácticos:

- Comprender o fluxo de información nunha interpolación de *strings* (`{{ }}`).
- Referenciar propiedades da compoñente e operar con elas dende o modelo HTML.
- Relacionar TS e vista definindo de tipos e propiedades entrambos.

Exemplo de enlazado de propiedades

Propoñemos agora un exemplo no que declaramos tres variables (`title`, que contén o título do noso proxecto, `documentacion` que é a URL da documentación de Angular, e `logo` que é a URL do logo de Angular) no noso TypeScript (por exemplo no `app.component.ts`):

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'enlazado-propiedades';
  documentacion: string = 'https://angular.io/docs';
  logo: string = 'https://angular.io/assets/images/logos/angular/logo-nav@2x.png';
}
```

Agora modificamos o arquivo da nosa vista (`app.component.html`) para visualizar o título con interpolación de *strings* coma fixemos anteriormente, incorporamos tanto a ligazón á documentación de Angular coma o logo mediante enlazado de propiedades.

```
<h1> {{ title }} </h1>

<a [href]="documentacion">Documentación de Angular</a>
<br>
<img [src]="logo" />
```

Finalmente visualizaremos algo semellante a isto:



Exemplo T05E04

Neste exemplo practícase o *binding* coñecido en Angular como enlazado de propiedades, o cal permite enviar información dende o controlador (TS) cara o modelo (HTML).

Mediante o desenvolvemento deste exemplo perséguese os seguintes obxectivos didácticos:

- Distinguir entre a interpolación de *strings* (`{{ }}`), que usamos para construír texto, e o enlazado de propiedades (`[ ]`), que usamos para dar valor a atributos do DOM.
- Enlazar URLs de maneira segura e tipada.
- Consolidar a relación TS-modelo definindo propiedades no TS e referenciándoas no HTML.

Exemplo de enlazado de eventos

Neste exemplo pretendemos capturar o `click` dun botón para mudar a cor da cabeceira `<h1>`. Os cambios realizados sobre o modelo HTML (**`app.component.html`**) foron os seguintes:

```
<h1 [ngStyle]="{ 'color': getCor() }">
  EXEMPLO DE ENLAZADO DE EVENTOS
</h1>
<button (click)="cambiarCor()">Cambiar cor do título</button>
```

Observamos que conectamos a cabeceira `<h1>` mediante enlazado de propiedades cun método `getCor()` implementado no TypeScript. Ao seu tempo o evento de `click` do botón asóciase ao método `cambiarCor()`. O arquivo TypeScript **`app.component.ts`** ficará como segue:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'enlazado-eventos';
  corPrincipal: string = "blue";

  // Este método permite mudar a cor principal definida
  cambiarCor(): void {
    if (this.corPrincipal == "blue") {
      this.corPrincipal = "violet";
    } else {
      this.corPrincipal = "blue";
    }
  }
}
```

```
// Este método devuelve a cor principal definida
getCor(): string {
  return this.corPrincipal;
}
}
```

O único que temos é unha variable definida co nome `corPrincipal` que muda a cor da cadea que contén en función de se o usuario fai clic no botón. Como se pode observar no capturador do evento, o método `cambiarCor()` muda o valor desa variable e amósaa por consola. O resultado será algo semellante a isto (inclúese a saída da consola na captura para entender mellor o funcionamento do exemplo):

EXEMPLO DE ENLAZADO DE EVENTOS

Cambiar cor do título

Exemplo T05E05

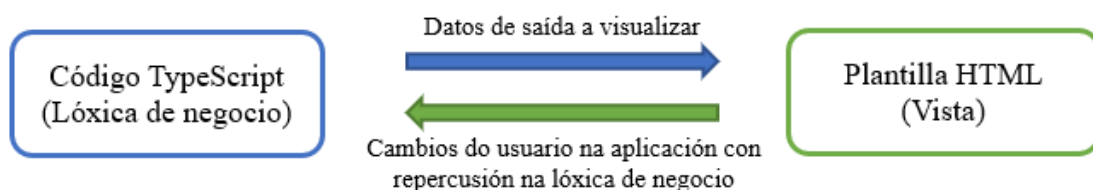
Este exemplo amosa como reaccionar a interaccións do usuario ligando eventos do modelo (HTML) con métodos da compoñente (TS), é dicir, enviando información dende o modelo cara ao controlador.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender o enlazado de eventos en Angular e como se emprega para chamar a métodos da compoñente.
- Visualizar a actualización reactiva da vista mediante propiedades de clase.
- Distinguir entre outros tipos de enlazado unidireccional para mostrar datos e enlazado de eventos (()) para manexar accións do usuario.

Enlazado bidireccional

Todos os métodos que vimos no apartado anterior de enlazado de datos unidireccional permiten facer cambios no modelo (TypeScript) e automaticamente están dispoñibles na vista (HTML). Porén, en ocasións podemos necesitar o contrario, é dicir, que os cambios na vista se reflexen no TypeScript, o que Angular posibilita a través do **enlazado bidireccional** ou *two-way binding*.



O enlazado bidireccional de Angular emprega a estrutura `[(ngModel)]="value"`, o que se coñece coma a «sintaxe da banana nunha caixa». No exemplo que sigue podemos ver como os datos da propiedade

`usuario.email` son empregados coma valor de entrada pero, se o usuario muda ese valor, a propiedade da compoñente é actualizada en consonancia automaticamente:

```
<input type="email" [(ngModel)]="usuario.email">
```

Para poder facer uso desta funcionalidade é preciso activar a directiva `ngModel` que depende do módulo `FormsModule` (para a interacción co usuario), e que podemos atopar no paquete `angular/forms`. Os cambios que temos que facer no noso código van depender de se traballamos en modo *standalone* (o modo por defecto dende Angular 17) ou se estamos a crear unha app por módulos.

Traballando en modo *standalone*

Unicamente teremos que incorporar `FormsModule` ao vector de importacións de cada compoñente que vaia empregar `[(ngModel)]`.

Traballando por módulos

Os cambios que hai que facer sobre o arquivo `app.module.ts` antes de poder empregar `ngModel` son os seguintes:

1. Facer as dúas importacións de `ngModule` e `FormsModule`.
2. Engadir no *array* de imports o módulo `FormsModule`.

O resultado será algo semellante ao seguinte:

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core'; // NOVA LIÑA
import { FormsModule } from '@angular/forms'; // NOVA LIÑA
import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule // NOVA LIÑA
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Exemplo de enlazado bidireccional

Facemos agora un pequeno cambio con respecto ao exemplo empregado anteriormente. O que queremos é engadir un `<input>` para que o usuario poida cambiar o título da nosa páxina, da nosa cabeceira `<h1>`. Polo

tanto, cando se cargue a páxina vaise amosar o valor que teña a cadea de título que definimos no TypeScript e, no caso de que o usuario introduza unha nova cadea no correspondente `<input>`, o título modificarase no instante. Este enlace bidireccional é posible grazas a `ngModel`.

A vista HTML (**`app.component.html`**) quedaría como segue:

```
<h1 [ngStyle]="{ 'color': getCor() }">
  {{ titulo }}
</h1>
<input type="text" [(ngModel)]="titulo" />
<br>
<button (click)="cambiarCor()">Cambiar cor do título</button>
```

No ficheiro **`app.component.ts`** simplemente engadimos unha variable co título inicial da cabeceira `<h1>`.

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, FormsModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'enlazado-bidireccional';
  titulo: string = "EXEMPLO DE ENLAZADO BIDIRECCIONAL";
  corPrincipal: string = "blue";

  // Este método permite mudar a cor principal definida
  cambiarCor(): void {
    if (this.corPrincipal == "blue") {
      this.corPrincipal = "violet";
    } else {
      this.corPrincipal = "blue";
    }
  }

  // Este método devolve a cor principal definida
  getCor(): string {
    return this.corPrincipal;
  }
}
```

Tras facer o cambio do título, escribindo no correspondente `<input>` xa se observa que en tempo real é modificada a vista no `<h1>` previo.

Programando en Angular

Programando en Angular

Cambiar cor do título

Exemplo T05E06

Este exemplo ilustra o uso do enlazado bidireccional mediante `[(ngModel)]` nun `<input>` que sincroniza a propiedade `title` co modelo. A vista reflicte o valor en tempo real con `<h1>{{ title }}</h1>` e inclúe un botón que invoca `resetearTitulo()` para restablecer o texto.

Os obxectivos didácticos procurados son os seguintes:

- Comprender o patrón `[(ngModel)]` e como sincroniza modelo (HTML) con controlador (TS) sen código *boilerplate*⁷.
- Ver a actualización inmediata do DOM ao cambiar o modelo e viceversa.
- Diferenciar o enlazado bidireccional (`[(ngModel)]`) dos distintos tipos de enlazado unidireccional de Angular: interpolación de *strings* (`{{ }}`), enlazado de propiedades (`[ ]`), enlazado de eventos (`( )`).
- Lembrar que `ngModel` require `FormsModule` (`imports` nun *standalone* ou no `NgModule`) para funcionar correctamente.

Erros e problemas habituais

A continuación algún dos erros e problemas que atoparemos habitualmente ao empregar enlazado de datos en Angular:

- **Erro: "Erro ao empregar `[(ngModel)]`".**
 - Causa: non se importou o módulo `FormsModule`.
 - Solución: engadir `FormsModule` no vector de importacións.
- **Erro: "Erro ao mostrar datos na vista"**
 - Causa: propiedade definida no TypeScript pero non inicializada.
 - Solución: asegúrate de inicializar a propiedade no constructor ou na declaración da clase.

```
export class NomeCompoñente {  
  texto: string = 'Texto de exemplo';  
}
```

⁷ Código boilerplate é aquel código repetitivo de tipo «estada» que se escribe para que outra cousa funcione (configuracións, importacións, clases ou arquivos base), pero que non achega lóxica específica do teu problema.

- **Erro: "Uncaught Error: Template parse errors"**
 - Causa: uso incorrecto de *binding* no HTML.
 - Solución: verifica que as expresións empreguen correctamente as claves `{{ }}` ou os corchetes `[ ]`.

Compoñentes

As **compoñentes** son a peza clave de Angular, encargadas de definir e controlar as diferentes «vistas» ou seccións visuais da aplicación.

Cada compoñente encapsula a súa propia estrutura (HTML), estilos (CSS) e comportamento (TS), promovendo unha separación clara de responsabilidades e funcionalidades.

Estrutura dunha compoñente

Unha compoñente represéntase mediante unha clase TypeScript que emprega o decorador `@Component` para engadir os seus metadatos. Estes metadatos definen a funcionalidade, aparencia e o xeito de integración da compoñente co resto da aplicación, sendo:

- **Selector (`selector`):**

- Define o nome da etiqueta HTML que representa á compoñente dentro do proxecto.
- Permite inserir a compoñente noutras vistas.
- Velaquí un exemplo dunha sección `selector` na cabeceira dunha compoñente:

```
selector: 'app-cabeceira'
```

O que permitiría empregar a compoñente mediante a etiqueta:

```
<app-cabeceira></app-cabeceira>
```

- **Modelo interno (`template`):**

- Contén o código HTML da vista da compoñente directamente escrito na propia cabeceira (*inline*).
- Velaquí un exemplo dunha sección `template` na cabeceira dunha compoñente:

```
template: '<h1>Benvid@s a Angular!</h1>'
```

- **Modelo externo (`templateUrl`):**

- Alternativo ao anterior, permite especificar o ficheiro externo onde está almacenado o código HTML da compoñente.
- Velaquí un exemplo dunha sección `templateUrl` na cabeceira dunha compoñente:

```
templateUrl: './cabeceira.component.html'
```

- **Estilos internos (`styles`):**

- Contén o código CSS da vista da compoñente directamente escrito na propia cabeceira (*inline*).
- Velaquí un exemplo dunha sección `styles` na cabeceira dunha compoñente:

```
styles: [  
  'h1 { color: blue; }'  
]
```

- **Estilos externos (`styleUrls`):**

- Contén a listaxe de ficheiros CSS que a compoñente emprega na súa vista.
- Velaquí un exemplo dunha sección `styleUrls` na cabeceira dunha compoñente:

```
styleUrls: [  
  './header.component.css'  
]
```

- **Encapsulamento (`encapsulation`):**

- Angular aplica por defecto a encapsulamento de estilos (Shadow DOM-like) para que o CSS dunha compoñente non interfira cos doutras.
- Velaquí un exemplo dunha sección `encapsulation` na cabeceira dunha compoñente:

```
encapsulation: ViewEncapsulation.Emulated
```

- **Anfitrión (`host`):**

- Define propiedades e eventos directamente no elemento que representa a compoñente.
- Velaquí un exemplo dunha sección `host` na cabeceira dunha compoñente:

```
host: {  
  '(click)': 'onClic()',  
  '[class.active]': 'isActive'  
}
```

Ademais dos metadatos anteriores, as compoñentes poden incorporar propiedades de entrada e saída para facilitar o intercambio de datos dende e cara á súa compoñente nai (que serán tratados en máis detalle nunha sección posterior):

- **Propiedades de entrada (`@Input`):** permiten á compoñente recibir datos dende a compoñente nai.
- **Propiedades de saída (`@Output`):** permiten á compoñente enviar datos cara á compoñente nai.

Creación de componentes

É habitual crear un directorio chamado **components** dentro de **src/app** para organizar as compoñentes do noso proxecto. Dentro dese directorio, recomendamos separar cada compoñente no seu propio cartafol. Esta práctica axuda a manter o proxecto limpo e ben estruturado.

Creación automática de compoñentes

Angular CLI permite crear compoñentes automaticamente empregando comandos específicos, o que simplifica o proceso. Velaquí os pasos:

1. Para crear unha compoñente chamada nova-componhente2 no cartafol **components**, executa:

```
ng generate component components/nova-componhente2
```

Ou, de forma abreviada:

```
ng g c components/nova-componhente2
```

O comando anterior creará automaticamente os seguintes ficheiros no cartafol correspondente:

- **nova-componhente2.component.ts**
- **nova-componhente2.component.html**
- **nova-componhente2.component.css**
- **nova-componhente2.component.spec.ts** (ficheiro de probas unitarias)

Ademais, a compoñente queda rexistrada automaticamente no proxecto, polo que podes usar o seu selector sen pasos adicionais.

Creación manual de compoñentes

Para crear unha compoñente manualmente tomamos os seguintes pasos:

1. Crea un novo cartafol en **src/app/components** chamado **nova-componhente**.
2. Dentro dese cartafol, crea un ficheiro chamado **nova-componhente.component.ts**.
3. Engade o seguinte código fonte no ficheiro:

```
import { Component } from '@angular/core';

@Component({
  selector: 'nova-componhente',
  standalone: true,
  imports: [],
  template: '<p>O contido da nova compoñente simplemente é un parágrafo.</p>',
  styles: ['p { font-weight: bold; color: green; }']
})
export class NovaComponhenteComponent {}
```

En proxectos máis grandes, separar o HTML e CSS en ficheiros independentes é unha boa práctica. Para facelo tomaríamos estes pasos:

4. Crea os ficheiros `nova-componhente.component.html` e `nova-componhente.component.css` no cartafol da compoñente.
5. Actualiza o decorador para referenciar eses ficheiros:

```
@Component({
  selector: 'nova-componhente',
  standalone: true,
  templateUrl: './nova-componhente.component.html',
  styleUrls: ['./nova-componhente.component.css']
})
```

Finalmente, para poder usar a compoñente no proxecto, precisamos incluíla no decorador `imports` doutra compoñente ou módulo. No noso caso haberá un único paso:

6. Modificamos o *script* da compoñente raíz (`app.component.ts`):

```
import { NovaComponhenteComponent } from './components/nova-componhente/nova-componhente.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [NovaComponhenteComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {}
```

Exemplo dunha compoñente

A continuación preséntase o ficheiro de comportamento (`cabeceira.component.ts`) unha compoñente chamada `CabeceiraComponent`:

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-cabeceira',
  templateUrl: './cabeceira.component.html',
  styleUrls: ['./cabeceira.component.css']
})
export class CabeceiraComponent {
  @Input() titulo: string;
  @Output() onLogout = new EventEmitter<void>();

  logout(): void {
    this.onLogout.emit();
  }
}
```

E de seguido podemos ver o seu código HTML (`cabeceira.component.html`):

```
<header>
  <h1>{{ titulo }}</h1>
  <button (click)="logout()">Logout</button>
</header>
```

E o seu código CSS (`cabeceira.component.css`):

```
header {
  background-color: #f4f4f4;
  padding: 10px;
  text-align: center;
}

button {
  margin-top: 10px;
}
```

A súa principal característica é que permiten estender funcións a partires doutras sen modificar a orixinal.

Exemplo T05E07

Este exemplo introduce o traballo con compoñentes en Angular: créase unha compoñente propia e intégrase na aplicación empregando o seu selector dentro da vista principal.

O exemplo persegue os seguintes obxectivos didácticos:

- Identificar as partes dunha compoñente: selector, *template*, estilos e clase TypeScript.
- Usar interpolación e enlazado de eventos básicos (p.ex. *click*) no template.
- Renderizar a compoñente secundaria dentro da principal mediante o seu selector.
- Comprender a integración no proxecto: declaración/importación da compoñente.

Exemplo T05E08

Este exemplo amosa como se dispoñen as compoñentes Angular na vista empregando *layouts* CSS para estruturar a páxina (p.ex.: cabeceira, barra lateral e contido principal).

Obxectivos didácticos:

- Comprender a separación entre estrutura (*templates* das compoñentes) e presentación (CSS de cada compoñente).
- Aplicar un *layout* CSS para organizar varias compoñentes na mesma vista.
- Ver como os estilos poden estar encapsulados na compoñente ou en follas globais, e cando usar cada opción.
- Integrar compoñentes «contedor» (*layout*) e compoñentes «presentacionais» (contido) mantendo responsabilidades claras.

Exemplo T05E09

Este exemplo amosa a composición e aniñamento de compoñentes *standalone* en Angular: a compoñente raíz renderiza á compoñente `CompoC` a cal, ao seu tempo, renderiza ás compoñentes `CompoA` e `CompoB` mediante os seus selectores e o *array imports* do decorador `@Component`.

Obxectivos didácticos

- Identificar a xerarquía de compoñentes dunha aplicación Angular.
- Comprender o uso de selectores para a renderización de compoñentes.
- Practicar a importación de compoñentes *standalone* no campo *imports* do decorador `@Component`.
- Comprender a delegación da vista dende a raíz cara a unha compoñente contedora.
- Distinguir a separación de responsabilidades: *template* e estilos propios en cada compoñente.

Erros e problemas habituais

A continuación algún dos erros e problemas que atoparemos habitualmente ao empregar compoñentes en Angular:

- **Erro: "Ficheiro ou estrutura de compoñente incompleta".**
 - **Causa:** o comando Angular CLI para crear a compoñente non se executou correctamente ou non se indicou o camiño desexado.
 - **Solución:** asegúrate de usar o comando correcto (e verifica que todos os ficheiros da compoñente foron creados correctamente):

```
ng generate component nome_da_compoñente
```

- **Erro: "Erro: 'nome-compoñente' is not a known element".**
 - **Causa:** a compoñente non foi declarada no módulo correspondente.
 - **Solución:** asegúrate de engadir a compoñente no vector `declarations` do módulo ou compoñente onde será empregada.

```
@NgModule({  
  declarations: [NomeCompoñente],  
  imports: [...],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

- **Erro: "Error: Cannot declare 'NomeCompoñente' in multiple modules".**
 - **Causa:** a compoñente foi declarada en máis dun módulo.
 - **Solución:** declara o compoñente nun único módulo ou utiliza un módulo compartido para exportalo.
- **Erro: "O compoñente non se renderiza no HTML".**
 - **Causa:** `selector` mal escrito ou non usado correctamente no HTML.
 - **Solución:** comproba o valor do `selector` no decorador `@Component` e usa o mesmo no ficheiro HTML.

```
@Component({  
  selector: 'app-nome-compoñente',  
  templateUrl: './nome-compoñente.component.html',  
  styleUrls: ['./nome-compoñente.component.css']  
})
```

- O uso correcto no HTML sería:

```
<app-nome-compoñente></app-nome-compoñente>
```

- **Erro: "Datos non dispoñibles na vista ao cargar o compoñente".**
 - Causa: os datos necesarios non se cargaron antes de renderizar o compoñente.
 - Solución: usa o *hook* `ngOnInit` para realizar a inicialización necesaria.

```
ngOnInit() {
  this.cargarDatos();
}
```

- **Erro: "Os estilos definidos non se aplican ao compoñente".**
 - Causa: o estilo foi definido de maneira global ou non está ben configurado no `styleUrls`.
 - Solución: asegúrate de que os estilos estean definidos no ficheiro CSS asociado e que `styleUrls` apunte correctamente a ese ficheiro.

```
styleUrls: ['./nome-compoñente.component.css']
```

- **Erro: "Problemas co cambio dinámico de contido na compoñente".**
 - Causa: cambios de estado mal xestionados ou non detectados.
 - Solución: usa `ChangeDetectorRef` para forzar a detección de cambios se é necesario.

Entrada de datos (o decorador @Input)

Angular pon á nosa disposición un decorador, **@Input**, que nos permite pasar información dende compoñentes nai cara compoñentes fillas (de xeito descendente na xerarquía de compoñentes).

Co gallo diso, márcase unha propiedade/atributo da compoñente nai co devandito decorador pasando a mesma a ser accesible polas compoñentes fillas.

Compoñente nai

Para o seguinte exemplo empregaremos coma compoñente nai á propia compoñente raíz de Angular (**app-root**). Ficando o seu código coma segue:

- **app.component.ts**: engadimos unha propiedade nova á cal chamamos **compartida**, neste caso de tipo numérico e iniciada co valor «0».

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';
import { FillaComponent } from '../filla/filla.component';
```

```

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, FillaComponent ],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'input';
  compartida : number = 0;
}

```

- **app.component.html:** neste ficheiro engadimos a etiqueta correspondente á compoñente filla (**app-filla**) cun *binding* de propiedades que asocia a súa propiedade **recibida** coa propiedade **compartida** da compoñente nai. A maiores incorporamos un botón que permite incrementar o valor da variable **compartida**, para comprobar como efectivamente a información flúe dende a compoñente nai cara á filla.

```

<app-filla [recibida]="compartida"></app-filla>
<button (click)="compartida=compartida+1">INCREMENTAR</button>

```

Compoñente filla

Supoñamos pois que temos unha compoñente filla denominada **nova-compoñente** co seguinte código:

- **filla.component.ts:** neste caso engadimos unha propiedade (**recibida**) á clase e acompañámola do decorador `@Input` para indicar que vai asociada á información recibida pola etiqueta HTML da compoñente. O valor asignado á propiedade unicamente serve para confirmar que efectivamente o valor inicial que aparecerá na páxina non será o que aquí figura, senón o que se reciba dende a compoñente nai.

```

import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-filla',
  standalone: true,
  imports: [],
  templateUrl: './filla.component.html',
  styleUrls: ['./filla.component.css']
})

export class FillaComponent {
  @Input() recibida : number = 10; // Poñemos un valor por defecto á propiedade
  para ver que efectivamente é sobreescrito dende a compoñente nai
}

```

- **filla.component.html:** o contido deste ficheiro é relativamente sinxelo, e unicamente hai unha interpolación de *strings* que dirixe á propiedade **recibida**.

```

<p>O valor da propiedade recibida por input é {{ recibida }}</p>

```

Exemplo T05E10

Este exemplo ilustra a comunicación de datos dende unha compoñente nai cara a unha filla empregando `@Input`.

O exemplo persegue os seguintes obxectivos didácticos:

- Comprender o fluxo de datos unidireccional de nai a filla con `@Input`.
- Aplicar *property binding* para pasar literais, expresións e obxectos á compoñente filla.
- Observar como reaccionar a cambios nos inputs (p.ex., `OnChanges`) para actualizar a vista.
- Adquirir boas prácticas: *inputs* pequenos e explícitos, evitar mutación directa de obxectos recibidos, separar presentación e estado.

Saída de datos (a función `output`)

A función auxiliar `output` de Angular permite o envío de información desde as compoñentes fillas cara ás compoñentes nai, é dicir, de xeito ascendente na xerarquía de compoñentes.

Para iso, defínese unha propiedade na compoñente filla empregando a función `output()`, o que a converte nun evento personalizado que pode ser escoitado pola compoñente nai mediante enlazado de eventos.

O valor emitido dende a compoñente filla pasa a estar dispoñible na compoñente nai como argumento do evento, permitindo a comunicación entre ambas dun xeito estritamente tipado.

Compoñente nai

No exemplo que figura a continuación empregaremos coma compoñente nai á propia compoñente raíz de Angular (`app-root`). O código será o seguinte:

- **`app.component.ts`**: engadimos unha propiedade chamada recibida (de tipo `string`) iniciada co valor «» e un método `amosarMensaxe(texto)` que actualiza o valor desa propiedade segundo o texto recibido desde a compoñente filla.

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { FillaComponent } from '../filla/filla.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, FillaComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'output';
  recibida: string = ''; // Valor inicial por defecto
```

```

    amosarMensaxe(texto: string): void {
        this.recibida = texto;
    }
}

```

- **app.component.html:** engadimos a etiqueta da compoñente filla (`app-filla`) cun *binding* de eventos que chama ao método `amosarMensaxe(texto)`, pasándolle como argumento o valor do evento.

```

<app-filla (mensaxe)="amosarMensaxe($event)"></app-filla>
<p>A compoñente filla enviou a seguinte mensaxe: <b>{{ recibida }}</b></p>

```

Compoñente filla

O código da nosa compoñente filla (**filla**) é como figura a continuación:

- **filla.component.ts:** neste caso engadimos unha propiedade chamada `mensaxe` que se define mediante a función `output()` con tipo `string`. Isto implica que se trata dun evento personalizado que pode ser emitido desde a compoñente e escoitado pola compoñente nai. Engadimos tamén un método `enviar()` no cal se emite dito evento empregando o método `.emit()` e pasando unha mensaxe como argumento.

```

import { Component, output, OutputEmitterRef } from '@angular/core';

@Component({
  selector: 'app-filla',
  standalone: true,
  imports: [],
  templateUrl: './filla.component.html',
  styleUrls: ['./filla.component.css']
})
export class FillaComponent {
  // Definimos o output usando a nova función output()
  mensaxe: OutputEmitterRef<string> = output<string>();

  // Emitimos o valor usando .emit()
  enviar(): void {
    this.mensaxe.emit("Ola mundo!");
  }
}

```

- **filla.component.html:** unicamente contén un botón HTML que chama ao método `enviar()`.

```

<button (click)="enviar()">ENVIAR</button>

```

Exemplo T05E11

Este exemplo ilustra a comunicación de datos dende unha compoñente filla cara a unha nai empregando a función `output()` e o obxecto `OutputEmitterRef`.

O exemplo persegue os seguintes obxectivos didácticos:

- Comprender o fluxo de datos de filla a nai mediante `output()` e o método `emit()`.
- Aplicar enlazado de eventos na nai para recibir o evento (`$event`) e actualizar o seu estado.
- Tipar correctamente o parámetros do `output<T>()` con parámetros escritos en TypeScript.
- Adoptar boas prácticas: nomes de eventos claros (p.ex., `valorActualizado`), emitir datos mínimos e inmutables, separar presentación (filla) e lóxica (nai).

Exemplo T05E12

Este exemplo ilustra a comunicación de datos bidireccional entre compoñentes nai e filla combinando o uso do decorador `@Input` e da función `output()`.

O exemplo persegue os seguintes obxectivos didácticos:

- Repasar o paso de datos do nai ao filla mediante `@Input`.
- Repasar o paso de datos de filla a nai mediante `output()` e `OutputEmitterRef`.
- Ver como manter un único estado «fonte de verdade» na nai e reflectilo na filla.
- Exercer o manexo de eventos do *template* e a actualización do estado na compoñente nai.

Directivas

As **directivas** de Angular son un dos elementos máis poderosos do *framework*, permitindo manipular o DOM de xeito modular e eficiente. Con elas, podemos modificar a estrutura do DOM, alterar a aparencia dos elementos ou engadir lóxica personalizada ao noso código baixo un nome predefinido.

Creación de Directivas

Para crear unha directiva personalizada empregamos o seguinte comando:

```
ng generate directive nome_da_directiva
```

Ou, empregando un *shorthand* propio de Angular CLI:

```
ng g d nome_da_directiva
```

Calquera dos comandos anteriores xera un ficheiro TypeScript cuxa finalidade é a de implementar a lóxica da directiva asegurando a súa integración no proxecto.

Tipos de directivas

En función do seu obxectivo destacamos tres tipos de directivas en Angular:

- **Directivas de compoñentes:** atópanse na clase principal e conteñen os detalles de como se procesa, instancia ou emprega a compoñente en tempo real.
- **Directivas estruturais:** empréganse para manipular e cambiar a estrutura da árbore DOM.

Destacamos tres das máis usuais:

- ***ngIf:** permite engadir ou eliminar un elemento DOM en función dunha condición lóxica.
- ***ngFor:** repite un determinado elemento HTML por cada elemento existente nunha lista iterable.
- ***ngSwitch:** permite engadir ou eliminar un elemento DOM en función dunha serie de valores que pode tomar unha variable.
- **ngPlural:** permite mostrar ou ocultar elementos en función dun valor numérico.
- **ngTemplate:** permite definir bloques de contido reutilizables sen renderizalos inmediatamente no DOM.
- **ngComponentOutlet:** permite renderizar compoñentes dinamicamente no DOM. Vincúlase a unha compoñente ou clase que debe ser instanciada e inserido no lugar onde se aplique a directiva.

- **Directivas de atributos:** cambian a vista e o comportamento dos elementos DOM. As dúas máis habituais son:
 - o `ngClass`: engade ou elimina clases CSS dun elemento HTML
 - o `ngStyle`: modifica o estilo dun elemento HTML empregando unha expresión que inclúe todos os cambios a realizar. Desta forma podemos cambiar dinamicamente o estilo dun elemento.
 - o `ngModel`: implementa o enlace bidireccional de datos entre un compoñente e un formulario.

A directiva `*ngIf`

Esta directiva é, xunto con `*ngFor`, posiblemente a máis empregada en Angular. Coma xa mencionamos con anterioridade, permite engadir ou eliminar un elemento HTML do DOM en base ao valor dunha expresión lóxica.

Para probar esta directiva imos aproveitar e usar a compoñente raíz **app** sobre a cal faremos as probas correspondentes.

Dentro do ficheiro **app.component.ts** podemos ver o nome que lle da ao selector que vai permitir empregar esa compoñente dende calquera vista en Angular. O selector correspondente (tamén coñecido coma «etiqueta»), neste caso, é a `app-root`.

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'ngIf';
}
```

Imos modificar a vista HTML **app.component.html** para probar a directiva `*ngIf` de xeito que permita amosar un elemento HTML en base a se o usuario fai clic nun botón ou non. Para iso imos reemplazar o seu contido co seguinte código:

```
<button (click)="amosar = !amosar ">
  {{amosar ? 'AGOCHAR' : 'AMOSAR'}}
</button>

O valor da variable amosar = {{amosar}}<br>
```



```
<h3 *ngIf="amosar">Elemento visible con *ngIf</h3>
```

Como pode verse, o que fixemos foi incorporar ao modelo anterior un botón no que se captura o evento de clic do usuario para, seguidamente, executar un código asociado ao mesmo. O enlace de eventos verémolo en profundidade máis adiante. O código o único que fai é gardar o valor da variable lóxica `show` negado (variable que estará na lóxica da compoñente ou, o que é o mesmo, no seu TypeScript asociado). É dicir, se `show` é verdadeiro ponse a falso e senón ao revés cando se fai un clic. No texto do botón temos as opcións de etiqueta `AGOCHAR` ou `AMOSAR`, escollidas en función do valor desa variable. A maiores amósase o valor da variable mediante o enlace de datos coas chaves `{{}}` (este enlace de datos tamén será analizado máis tarde en profundidade). Finalmente na etiqueta `<h3>` é onde facemos uso da nosa directiva `*ngIf`, de xeito que se amose ou non o propio elemento en función do valor de `amosar`.

Retornamos ao ficheiro `app.component.ts` para incluír a declaración da variable `amosar`, o cal realizaremos dentro da clase que lle corresponde, producíndose a inicialización dentro do construtor (neste caso co valor de verdadeiro).

```
...
export class AppComponent {
  title = 'ngIf';
  amosar: boolean;

  constructor() {
    this.amosar=true;
  }
  ...
}
```

Agora podemos visualizar no noso navegador o resultado nas súas dúas formas:

AGOCHAR O valor da variable show = true
Elemento visible con *ngIf

AMOSAR O valor da variable show = false

Exemplo T05E13

Neste exemplo demostrase o uso básico da directiva `*ngIf`: un botón conmuta a variable booleana `cargar` e, en función do seu valor, insírese ou elimínase do DOM un encabezado. Tamén se amosa na vista o valor actual de `cargar` para observar o cambio en tempo real, empregando `(click)="cargar = !cargar"` e a condición `*ngIf="cargar"`.

Obxectivos didácticos

- Comprender que `*ngIf` crea/destrúe elementos no DOM segundo unha condición (non os oculta).
- Practicar o enlazado de eventos para actualizar estado dende a vista.
- Observar a actualización inmediata mediante interpolación do valor de `cargar`.
- Lembrar que `*ngIf` pertence a `CommonModule` e debe estar dispoñible no compoñente.

Exemplo T05E14

Este exemplo amosa o uso combinado de `*ngIf` cunha compoñente: un botón conmuta a variable booleana `cargar` e, segundo o seu valor, insírese ou elimínase do DOM a compoñente filla. Na vista móstrase tamén o estado actual de `cargar` para observar o comportamento.

Obxectivos didácticos perseguidos con este exemplo:

- Comprender como `*ngIf` controla a creación e destrución de compoñentes no DOM.
- Practicar o enlazado de eventos para modificar estado (clic que conmuta `cargar`).
- Relacionar estado booleano, renderización condicional e ciclo de vida da compoñente.

A directiva `*ngFor`

Esta directiva permite repetir unha serie de elementos HTML tantas iteracións como teña o percorrido dunha lista iterable (`Collection`). Existen unha serie de variables locais tales como `Index`, `First`, `Last`, `odd` e `even` que son exportadas por esta directiva e que posibilitan e asinxelan notablemente o seu manexo.

Facendo uso da compoñente raíz (**app**), imos crear un exemplo cun `array` definido no arquivo TypeScript no cal introduciremos catro elementos. Estes elementos vanse amosar no modelo HTML facendo uso de `*ngFor`. A maiores incluiremos no noso arquivo CSS algúns nomes de clases para introducir tamén o uso de `ngClass`, o cal vai posibilitar incorporar ou eliminar novas clases CSS nos elementos HTML.

No arquivo **app.component.ts** definimos a listaxe de elementos, que será inicializada no construtor:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'ngFor';
  provincias: string[];

  constructor() {
    this.provincias = ["A Coruña", "Lugo", "Ourense", "Pontevedra"];
  }
}
```

Incluimos agora algunha clase no ficheiro CSS **app.component.css** para modificar a cor de texto dalgún dos elementos:

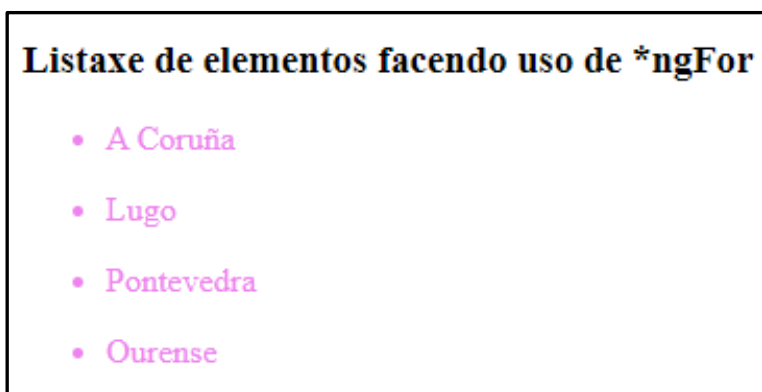
```
.textoVioleta {
  color: violet;
}

.textoVerde {
  color: green;
}
```

Pola súa parte, no arquivo **app.component.html** empregamos `*ngFor` indicando un nome de variable item que en cada iteración acollerá un elemento e permitirá introduci-lo como un novo `<li>` usando a clase `textoVioleta` (a clase `textoVerde`) será empregada en exemplos posteriores.

```
<h1>Listaxe de elementos facendo uso de *ngFor</h1>
<ul>
  <li *ngFor="let provincia of provincias" ngClass="textoVioleta">
    {{provincia}}
  </li>
</ul>
```

Ao lanzarmos a aplicación, o resultado será algo así:

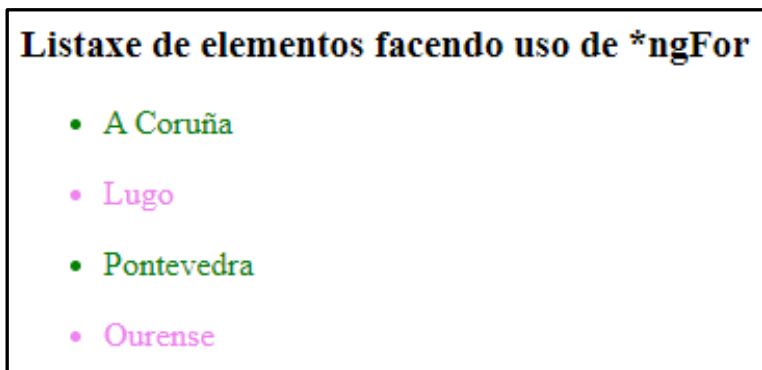


Podemos levar o noso exemplo un bocadiño alén, empregando algunha das variables propias que exporta esta directiva. Por exemplo, se queremos que os elementos pares teñan unha cor e os impares outra, teremos que empregar as variables `odd` e `even` para impares e pares respectivamente.

Modificando o noso código `probasngfor.component.html` por:

```
<h3>Listaxe de elementos facendo uso de *ngFor</h3>
<ul>
  <li
    *ngFor="let provincia of provincias; let impar=odd; let par=even;" [ngClass]="{'textoVioleta': impar, 'textoVerde': par}">
    {{provincia}}
  </li>
</ul>
```

Veremos un resultado semellante ao seguinte:



En cada iteración gárdanse, polo tanto, dúas variables (`par` e `impar`) de tipo booleano que son avaliadas por `ngClass`, que indicará que clase aplicará en función das mesmas. Se nos fixamos, aparece un elemento imprescindible que son os corchetes, os cales conteñen a directiva `ngClass`, e encárganse de facer o *binding*, indicando se existe algún elemento debe ser substituído polo seu valor (isto será analizado con máis detalle no apartado de enlace de datos).

Exemplo T05E15

Este exemplo ilustra como xerar listas dinámicas na vista empregando a directiva estrutural `*ngFor`, iterando sobre un *array* de cadeas para pintar cada elemento nun `<li>`.

O exemplo persegue estes obxectivos didácticos:

- Comprender a sintaxe de `*ngFor (let item of coleccion)` e o seu lugar no *template*.
- Diferenciar entre datos da compoñente e representación na vista mediante interpolación.
- Observar como a lista se actualiza automaticamente ao engadir, eliminar ou reordenar elementos do *array*.

Exemplo T05E16

Neste exemplo móstrase unha listaxe de equipos dunha división empregando `*ngFor` e interpolación. Un botón lanza a función de compoñente que muda a división activa e actualiza a listaxe, amosando a relación entre estado do compoñente, enlazado de eventos e renderizado reactivo do *template*.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender o uso de `*ngFor` para iterar e pintar coleccións.
- Observar como un evento de usuario actualiza o estado e a vista.
- Relacionar interpolación, enlazado de eventos e cambio de estado nun fluxo único.

Exemplo T05E17

Neste exemplo móstrase como empregar `*ngFor` para instanciar dinamicamente unha compoñente personalizada por cada elemento dunha listaxe, pasando datos ao fillo mediante `@Input` e organizando a saída cunha grade sinxela.

Obxectivos didácticos que persegue este exemplo:

- Comprender como `*ngFor` crea múltiples instancias dunha compoñente a partir dun *array*.
- Practicar o paso de datos do pai ao fillo con `@Input` dentro da iteración.
- Identificar os requisitos mínimos: compoñente *standalone*/importada, selector correcto e dispoñibilidade de `*ngFor`.
- Diferenciar o renderizado en bucle de elementos nativos fronte a compoñentes personalizadas.

A directiva `*ngSwitch`

Trátase tamén dunha directiva estrutural, que neste caso permite eliminar e engadir múltiples elementos do DOM en función dos valores que poida tomar unha determinada expresión ou variable.

Imos crear, coma exemplo, unha nova compoñente na cal dispoñemos dunha variable numérica, de xeito que se o número ten un valor entre 1 e 6, vai amosarse o elemento correspondente dunha listaxe. No caso de que o número sexa outro, indicárase que non se atopa ningún elemento en correspondencia. Ademais do anterior, empregaremos un botón, de xeito que cada vez que se faga clic no mesmo aumente o valor numérico da variable nunha unidade un elemento, coa condición de que se o valor é maior que 6 entón volverá ser 1.

Os elementos principais desta directiva son tres:

- **[ngSwitch]**: onde avaliamos o valor dunha expresión ou variable.
- ***ngSwitchCase**: que corresponde co caso concreto no que a variable teña ese valor indicado.
- ***ngSwitchDefault**: que corresponde co caso por defecto cando o valor da variable non emparellou con ningún dos casos definidos anteriormente.

O código da vista HTML (`app.component.html`) será o seguinte:

```
<h3>Elemento amosado en función de "elem" mediante a directiva *ngSwitch</h3>

<button (click)="elem>6 ? elem=1 : elem=elem+1">Incrementar NUM elemento</button>

<div [ngSwitch]="elem">
  <p *ngSwitchCase="1" [ngStyle]="{'color': 'green'}">0 seleccionado
  foi {{listaxe[0]}}</p>
  <p *ngSwitchCase="2" [ngStyle]="{'color': 'red'}">0 seleccionado
  foi {{listaxe[1]}}</p>
  <p *ngSwitchCase="3" [ngStyle]="{'color': 'blue'}">0 seleccionado
  foi {{listaxe[2]}}</p>
  <p *ngSwitchCase="4" [ngStyle]="{'color': 'orange'}">0 seleccionado
  foi {{listaxe[3]}}</p>
```

```

    <p *ngSwitchCase="5" [ngStyle]="{'color': 'yellow'}">O seleccionado
foi {{listaxe[4]}}</p>
    <p *ngSwitchCase="6" [ngStyle]="{'color': 'violet'}">O seleccionado
foi {{listaxe[5]}}</p>
    <p *ngSwitchDefault [ngStyle]="{'color': 'gray'}">Non é un elemento da
listaxe</p>
</div>

```

O código do TypeScript (**app.component.ts**) é o seguinte:

```

import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'ngSwitch';
  numero : number;
  listaxe : string[];

  constructor() {
    this.numero = 0;
    this.listaxe = ["DAW", "DAM", "ASIR", "SMIR"];
  }
}

```

Un dos resultados intermedios de operar o proxecto anterior será algo semellante a isto:

Elemento amosado en función de "elem" mediante a directiva *ngSwitch

Incrementar NUM elemento

O seleccionado foi ELEMENTO 3

Exemplo T05E18

Este exemplo amosa o uso de `*ngSwitch` para renderizar distintos parágrafos en función do valor numérico de `elem`. Un botón incrementa `elem` (nun ciclo que vai do 1 ao 6) e cada caso amosa o elemento correspondente da listaxe (`continentes`). Se o valor non coincide con ningún caso, amósase a rama por defecto.

Os obxectivos didácticos son:

- Comprender a sintaxe e o fluxo de `*ngSwitch`, `*ngSwitchCase` e `*ngSwitchDefault`.
- Relacionar o estado da interface coas propiedades do compoñente e índices de *arrays*.
- Evitar cadeas de `*ngIf` empregando unha estrutura de selección clara.
- Practicar a actualización de estado dende eventos (*click*) e a súa repercusión na vista.

A directiva ngPlural

Renderiza elementos DOM segundo valores numéricos. O anterior acádase avaliando o valor dunha variable ou expresión programática.

Neste exemplo podemos ver como se avalía o valor da variable `numero` para cargar distintos modelos no HTML:

```
<div [ngPlural]="numero">
  <ng-template ngPluralCase="1">Un elemento</ng-template>
  <ng-template ngPluralCase="other">{{ numero }} elementos</ng-template>
</div>
```

Exemplo T05E19

Exemplo básico de `ngPlural` que mostra mensaxes diferentes segundo o valor dunha variable numérica (`contador`) e actualiza a vista ao incrementar e decrementar o valor da mesma dende a interface.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender o funcionamento de `ngPlural` e a súa sintaxe declarativa con `ngPluralCase`.
- Diferenciar casos específicos (`=0`, `=1`, `=2` etc.) do caso comodín (`other`).
- Relacionar cambios de estado na compoñente con actualizacións reactivas na vista.

A directiva ngTemplate

Esta directiva permite definir modelos que formarán código reutilizable.

Vexamos un exemplo de definición dun modelo e a súa posterior aplicación condicionada a través dunha directiva `*ngIf`.

```
<ng-template #modelo>
  <p>Este contido pode ser reutilizado.</p>
</ng-template>

<div *ngIf="true; then modelo"></div>
```

Exemplo T05E20

Este exemplo amosa como definir un `ng-template` e inxectar datos mediante un contexto, diferenciando entre o valor implícito (`$implicit`) e as propiedades nomeadas. Emprega `let-nome` (para capturar o parámetro nome de maneira implícita) e `let-prezo="prezo"` para mapear unha clave do contexto, permitindo reutilizar a mesma vista con distintos valores.

Este exemplo ten os seguintes obxectivos didácticos:

- Comprender a diferenza entre `$implicit` e propiedades nomeadas ao pasar contexto a `ng-template`.
- Aprender o uso de `let-parametro` e `let-parametro="clave"` para recibir datos dentro do template.
- Practicar a inserción dun *template* con `*ngTemplateOutlet` e a creación do seu `context`.
- Fomentar a reutilización de vistas parametrizables sen necesidade de crear compoñentes adicionais.

A directiva `*ngComponentOutlet`

Esta directiva permite renderizar compoñentes dinamicamente.

De seguido vemos un exemplo práctico:

```
<ng-container *ngComponentOutlet="compoñente"></ng-container>
```

Exemplo T05E21

Este exemplo amosa como cargar compoñentes de forma dinámica en tempo de execución empregando a directiva estrutural `*ngComponentOutlet`, o que permite alternar entre varias vistas sen rutas nin `*ngIf` encadeados.

Obxectivos didácticos que se perseguen con este exemplo:

- Comprender que `*ngComponentOutlet` recibe unha referencia á clase da compoñente e non o seu selector.
- Aprender a cambiar a compoñente activa dende a lóxica da aplicación para renderizar dinamicamente diferentes vistas.
- Recoñecer casos de uso típicos: contedores dinámicos, demos, *widgets* configurables e probas de interface.

A directiva `ngClass`

Esta directiva aplica ou elimina clases CSS dinamicamente, permitindo asignar estilos en base ao comportamento (TS) dunha compoñente.

Agora vexamos un exemplo práctico de uso da mesma:

```
<p [ngClass]="{ 'activo': estadoActivo, 'inactivo': !estadoActivo }">
  Estado: {{ estadoActivo ? 'Activo' : 'Inactivo' }}
</p>
```

Exemplo T05E22

Este exemplo amosa como aplicar clases CSS dinamicamente con `ngClass` mentres se itera unha listaxe mediante `*ngFor`. Empregamos as variables de contexto `odd` e `even` para alternar estilos segundo a paridade da posición de cada elemento da listaxe.

Obxectivos didácticos do exemplo:

- Comprender a sintaxe obxecto de `ngClass` para engadir clases en función de condicións booleanas.
- Usar as variables de contexto de `*ngFor` (`odd/par`, `even/impar`) para establecer estilos alternos.
- Separar claramente lóxica (datos e condicións) e presentación (clases CSS) para manter un código limpo e reutilizable.

A directiva `ngStyle`

Modifica estilos *inline* de maneira condicional.

Vexamos tamén neste caso un exemplo de aplicación:

```
<p [ngStyle]="{ 'color': estado ? 'green' : 'red' }">
  Estado: {{ estado ? 'Activo' : 'Inactivo' }}
</p>
```

Exemplo T05E23

Este exemplo mostra como aplicar estilos en liña con `ngStyle` en función do valor dunha variable controlada por `*ngSwitch`, cambiando a cor do texto para cada caso.

A través deste exemplo perseguimos estes obxectivos didácticos:

- Comprender a sintaxe en forma de obxecto que usa `ngStyle` para establecer estilos en tempo de execución.
- Aplicar estilos condicionais segundo o valor de `elem` empregando `*ngSwitch` e `*ngSwitchCase`.
- Combinar directivas estruturais (`ngSwitch`) con directivas de atributo (`ngStyle`) mantendo a lóxica na clase e a presentación na vista.
- Valorar cando usar `ngStyle` fronte a clases CSS (`ngClass`) para cambios simples e directos de estilo.

Exemplo T05E24

Neste exemplo contrástase o uso de `ngClass` (aplicación de clases CSS segundo condicións) fronte a `ngStyle` (estilos en liña condicionais) nun mesmo escenario: un botón conmuta o estado do sistema entre «Activo» e «Inactivo» modificando as propiedades `activo` e `estado`.

Perseguiamos, a través do exemplo, estes obxectivos didácticos:

- Diferenciar cando empregar `ngClass` (reutilización de estilos e mantemento en CSS) e cando `ngStyle` (cambios puntuais en liña).
- Aplicar estilos condicionais baseados en booleanos: clases `activo/inactivo` con `ngClass` e cores dinámicas con `ngStyle`.
- Entender a interacción entre lóxica de compoñente (`mudarEstado`, `activo`, `estado`) e a vista para reflectir cambios de UI de forma reactiva.

A directiva ngModel

A directiva `ngModel` implementa o enlace bidireccional entre datos e formularios. Esta directiva xa foi tratada no apartado relativo ao enlazado de datos.

Directivas Personalizadas

As directivas personalizadas amplían a funcionalidade dos elementos HTML para adaptarse ás necesidades específicas do desenvolvemento.

No seguinte exemplo podemos ver a aplicación dunha directiva personalizada que permite ocultar elementos en base a un valor booleano:

```
@Directive({
  selector: '[appOcultar]'
})
export class OcultarDirective {
  @Input('appOcultar') set mostrar(valor: boolean) {
    this.el.nativeElement.style.display = valor ? 'block' : 'none';
  }

  constructor(private el: ElementRef) {}
}
```

E a súa aplicación a través do *template* HTML:

```
<div [appOcultar]="true">Este elemento está visible.</div>
```

A continuación podemos ver un segundo exemplo de creación de directivas personalizadas. Neste caso creamos unha directiva que resalta un texto ao pasar o rato por riba do mesmo:

- Implementamos a lóxica no ficheiro **destacar.directive.ts**:

```
import { Directive, ElementRef, HostListener } from '@angular/core';

@Directive({
  selector: '[appDestacar]'
})
export class DestacarDirective {
  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.cambiarCor('yellow');
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.cambiarCor('');
  }
}
```

```
private cambiarCor(color: string) {  
  this.el.nativeElement.style.backgroundColor = color;  
}  
}
```

- Aplicación da directiva:

```
<p appDestacar>Pasa o rato por riba deste texto para resaltalo.</p>
```

Exemplo T05E25

Neste exemplo créanse dúas directivas de atributo: unha para controlar a visibilidade dun elemento a partir dun booleano, e outra para aplicar estilos a un elemento ao pasar o rato sobre o mesmo, admitindo unha cor opcional e usando “amarelo” por defecto cando o atributo aparece sen valor. Ilústrase o uso de `@Directive`, entradas con `@Input` (incluído un *setter* con valor por defecto), xestión de eventos con `@HostListener` e aplicación de estilos mediante `Renderer2`.

O exemplo ten estes obxectivos didácticos:

- Comprender como modificar presentación e comportamento de elementos mediante directivas de atributo.
- Empregar `@Input` con validación e valores por defecto nunha directiva.
- Manexar eventos do rato dentro da directiva con `@HostListener`.
- Aplicar estilos con `Renderer2` seguindo boas prácticas de Angular.

Erros e problemas habituais

A continuación algún dos erros e problemas que atoparemos habitualmente ao empregar directivas en Angular:

- **Erro: "Directiva personalizada non funciona".**
 - Causa: `selector` mal definido no decorador `@Directive`.
 - Solución: verificar o nome do `selector` e asegurar que está usado correctamente no HTML.
- **Erro: "Non se renderiza unha compoñente con `ngComponentOutlet`".**
 - Causa: compoñente non importado ou mal configurado.
 - Solución: revisar as importacións no módulo ou compoñente raíz.

Decoradores

Un **decorador** é un patrón de deseño empregado para modificar o comportamento dun elemento (clase, propiedade, método ou parámetro) sen necesidade de modificar o seu código fonte. Dito doutro xeito, un decorador engade funcionalidade dinamicamente a un obxecto evitando ter que crear subclases.

Mediante o uso de decoradores, as compoñentes son rexistradas, de xeito que poden ser empregadas noutras partes da aplicación.

Tipos de decoradores

Angular contempla catro tipos principais de decoradores:

- **De clase:** unha función que recibe a clase coma parámetro devolvendo seguidamente algo que facer coa mesma. P.ex.: `@Component` e `@NgModule`.
- **De propiedades:** unha función que recibe coma parámetros a clase e o nome da propiedade. P.ex.: `@Input` e `@Output`.
- **De métodos:** unha función que recibe como parámetros a clase, o nome do método e un descritor da propiedade do obxecto⁸. P.ex.: `@HostListener`.
- **De parámetros:** unha función que recibe coma parámetros a clase, o nome do parámetro e a súa posición. P.ex.: `@Inject`.

Coma podemos ver polos exemplos anteriores, os nomes dos decoradores van sempre precedidos dunha arroba (@) seguida do nome do elemento ao cal decoran.

Decoradores de clases

O decorador `@Injectable`

O decorador `@Injectable` en Angular úsase para marcar unha clase como dispoñible para a inxección de dependencias no sistema de Angular. Este decorador indica que a clase pode ser inxectada noutros compoñentes, servizos ou módulos.

A continuación o código TS dun exemplo:

```
import { Injectable } from '@angular/core';
```

⁸ Un descritor da propiedade do obxecto é un método estático que define unha propiedade directamente sobre un obxecto ou ben modifica directamente unha propiedade existente do obxecto para, a continuación, devolver o devandito obxecto.

```
@Injectable({
  providedIn: 'root'
})
export class ExemploServico {
  obterDados() {
    return 'Datos dende o servizo.';
  }
}
```

Decoradores de propiedades

O decorador @HostBinding

O decorador **@HostBinding** en Angular úsase para ligar propiedades de clase dunha directiva ou compoñente con propiedades, atributos ou estilos do elemento anfitrión (*host*). Dese xeito, permite modificar directamente o comportamento ou aparencia dun elemento HTML dende a lóxica da clase. Dito doutro xeito, ao mudar o valor da propiedade na clase (da directiva ou compoñente), Angular actualizará o DOM do elemento automaticamente.

Imos agora co código TS dun exemplo de aplicación desta directiva:

```
import { Directive, HostBinding } from '@angular/core';

@Directive({
  selector: '[appLigado]'
})
export class LigadoDirective {
  @HostBinding('style.color') corTexto = 'red';
}
```

E o código do *template* HTML:

```
<p appLigado>Este texto está ligado ao atributo de estilo.</p>
```

Exemplo T05E26

Neste exemplo úsase unha directiva *standalone* para ligar, mediante `@HostBinding`, a propiedade `style.color` do elemento anfitrión a unha propiedade da clase, aplicando a cor ao texto directamente desde a lóxica da directiva.

Este exemplo persegue os seguintes obxectivos didácticos:

- Comprender que é o elemento anfitrión e como `@HostBinding` liga propiedades da clase a propiedades/atributos/estilos do *host*.
- Aplicar un estilo simple (cor do texto) dende a directiva sen manipular o DOM nin empregar `nativeElement`.
- Integrar a directiva *standalone* nunha vista e comprobar visualmente o efecto da ligazón.

Decoradores de métodos

O decorador `@HostListener`

O decorador `@HostListener` en Angular úsase para escoitar eventos do DOM directamente no elemento anfitrión (*host*) dunha directiva ou compoñente. Permite asociar métodos de clase a eventos específicos, como `click`, `mouseover`, ou `keyup`, de forma que se executen automaticamente cando o evento ocorra.

Velaquí un exemplo práctico de uso desta directiva. Comezamos polo *script TS*:

```
import { Directive, HostListener } from '@angular/core';

@Directive({
  selector: '[appClic]'
})
export class ClicDirective {
  @HostListener('click', ['$evento'])
  aoClicar(evento: Event) {
    console.log('Elemento clicado:', evento);
  }
}
```

E no *template* HTML engadiríamos:

```
<p appClic>Fai clic neste parágrafo para activar o evento.</p>
```


Exemplo T05E27

Neste exemplo ilústrase como escoitar eventos do rato directamente na clase mediante `@HostListener` (`mouseenter` e `mouseleave`) e reaccionar cambiando o estilo dun elemento referenciado con `ViewChild`. A lóxica de evento fica encapsulada na clase (sen enlazado de eventos no *template*) e emprégase un método auxiliar para aplicar a cor de fondo.

Obxectivos didácticos do exemplo:

- Comprender como `@HostListener` liga eventos do elemento anfitrión a métodos da clase.
- Diferenciar o uso de `@HostListener` fronte ao enlazado de eventos no *template*.
- Empregar `ViewChild` para referenciar un elemento do DOM e modificar o seu estilo de forma controlada.
- Organizar a lóxica de resposta a eventos en métodos reutilizables (p. ex. `mudarCor`).

Exemplo T05E28

Neste exemplo utilízase `@HostListener` para escoitar eventos globais do documento e da xanela: pulsacións de teclas (`Enter` e combinación `Shift+Q`), clics no documento e desprazamentos da páxina. Cada evento actualiza un contador amosado na vista, exemplificando como reaccionar a interaccións do usuario sen enlazado directo no *template*.

Obxectivos didácticos

- Comprender a sintaxe de `@HostListener` para eventos con ámbitos de orixe distintos (`document`, `window`) e combinacións de teclas.
- Distinguir entre manexo de eventos no *template* e na clase, centralizando a lóxica en métodos.
- Actualizar estado do compoñente en resposta a eventos de teclado, rato e *scroll* e reflectilo na vista.

Decoradores de parámetros

O decorador `@Inject`

O decorador `@Inject` en Angular úsase para realizar a inxección de dependencias manualmente. Permite especificar un *token* ou un provedor específico que debe ser inxectado nunha compoñente, servizo, ou outro elemento, mesmo en situacións nas que Angular non pode deducilo automaticamente.

Vexamos un exemplo disto:

```
import { Component, Inject } from '@angular/core';
import { DOCUMENT } from '@angular/common';

@Component({
  selector: 'app-inxeccion',
  template: '<p>Compoñente con inxección de dependencias.</p>'
})
export class InxeccionComponent {
  constructor(@Inject(DOCUMENT) private documento: Document) {
    console.log('Documento inxectado:', this.documento);
  }
}
```

O decorador @Directive

O decorador **@Directive** en Angular úsase para definir unha directiva personalizada. Permite engadir comportamento específico aos elementos do DOM, modificando o seu aspecto, estilo ou funcionalidade.

E aquí un exemplo de aplicación da mesma, comezando polo código TS:

```
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appDestacar]'
})
export class DestacarDirective {
  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

E o código HTML:

```
<p appDestacar>Este texto ten unha cor de fondo destacada.</p>
```

Erros e problemas habituais

A continuación algún dos erros e problemas que atoparemos habitualmente ao empregar decoradores en Angular:

- **Erro: "Erro ao empregar @Injectable".**
 - Causa: configuración incorrecta do ámbito no decorador.
 - Solución: comprobar se `providedIn` está correctamente definido.
- **Erro: "Non se reciben datos na compoñente".**
 - Causa: o decorador `@Input` non está configurado correctamente.

- Solución: comproba que o compoñente pai está enviando datos coa sintaxe axeitada.

```
@Input() dato: string = '';
```

- **Erro: "Os eventos da compoñente non son capturados".**
 - Causa: o decorador `@Output` non está configurado ou conectado correctamente.
 - Solución: usa `EventEmitter` para emitir eventos dende o compoñente fillo e asegúrate de que o compoñente pai os captura.

```
@Output() evento = new EventEmitter<string>();

emitirEvento() {
  this.evento.emit('Mensaxe enviada');
}
```

Bibliografía

- «El gran libro de Angular». (2018). Boada, M. e Gómez, J.A. Marcombo (Barcelona).
- Decoradores en TypeScript: <https://rafaelneto.dev/blog/decoradores-typescript/>
- Documentación oficial de Angular: <https://angular.io/docs>
- Documentación oficial de **Node.js**: <https://nodejs.org/es/docs/>
- Instalar **Angular CLI** en Ubuntu 20.04 LTS: <https://noviello.it/es/como-instalar-angular-cli-en-ubuntu-20-04-lts/>
- Instalar **Node.js** en Ubuntu 20.04 LTS: <https://www.digitalocean.com/community/tutorials/how-to-install-node-js-on-ubuntu-20-04-es>
- Página oficial de **Angular**: <https://angular.io/>
- Página oficial de **Node.js**: <https://nodejs.org/es/>

ÍNDICE

INTRODUCCIÓN A ANGULAR	1
ARQUITECTURA DE ANGULAR 2+.....	1
TYPESCRIPT.....	2
O PATRÓN MVC.....	4
INSTALACIÓN E POSTA A PUNTO	6
INSTALACIÓN DE NODE.JS	6
INSTALACIÓN DE ANGULAR CLI	7
PRIMEIROS PASOS	8
CREACIÓN DUN PROXECTO ANGULAR	8
ESTRUTURA DUN PROXECTO ANGULAR	9
DESPREGAMENTO DE APLICACIÓNS ANGULAR	12
INICIO E CARGA DUN PROXECTO EN ANGULAR	14
OPTIMIZACIÓNS NO PROCESO DE CARGA	15
A NOSA PRIMEIRA APLICACIÓN ANGULAR.....	15
ENLAZADO DE DATOS	16
ENLAZADO UNIDIRECCIONAL.....	16
ENLAZADO BIDIRECCIONAL	20
ERROS E PROBLEMAS HABITUAIS.....	23
COMPOÑENTES	25
ESTRUTURA DUNHA COMPOÑENTE	25
CREACIÓN DE COMPOÑENTES	27
EXEMPLO DUNHA COMPOÑENTE	28
ERROS E PROBLEMAS HABITUAIS.....	31
ENTRADA DE DATOS (O DECORADOR @INPUT)	32
SAÍDA DE DATOS (O DECORADOR @OUTPUT).....	34
DIRECTIVAS	37
CREACIÓN DE DIRECTIVAS	37
TIPOS DE DIRECTIVAS	37
A DIRECTIVA *NGIF	38
A DIRECTIVA *NGFOR	40

A DIRECTIVA *NGSWITCH.....	44
A DIRECTIVA NGPLURAL	46
A DIRECTIVA NGTEMPLATE.....	46
A DIRECTIVA *NGCOMPONENTOUTLET	47
A DIRECTIVA NGCLASS	48
A DIRECTIVA NGSTYLE	49
A DIRECTIVA NGMODEL	50
DIRECTIVAS PERSONALIZADAS.....	50
ERROS E PROBLEMAS HABITUAIS.....	51
DECORADORES	52
TIPOS DE DECORADORES	52
DECORADORES DE CLASES	52
DECORADORES DE PROPIEDADES	53
DECORADORES DE MÉTODOS	54
DECORADORES DE PARÁMETROS	55
ERROS E PROBLEMAS HABITUAIS.....	56
BIBLIOGRAFÍA.....	58